ch10-UWFLecture-BAJ

January 11, 2025

1 Chapter 10 - Clustering

M7 Lecture Codes Adopted from Sebastian Raschka Github and commented and slightly revised by Dr. Brian Jalaian and Johnny Yu

---

1.1 Table of Contents

---

[29]:
```
from IPython.display import Image
%matplotlib inline
```

1.2 10.1.Grouping objects by similarity using k-means

1.2.1 10.1.1.K-means clustering using scikit-learn

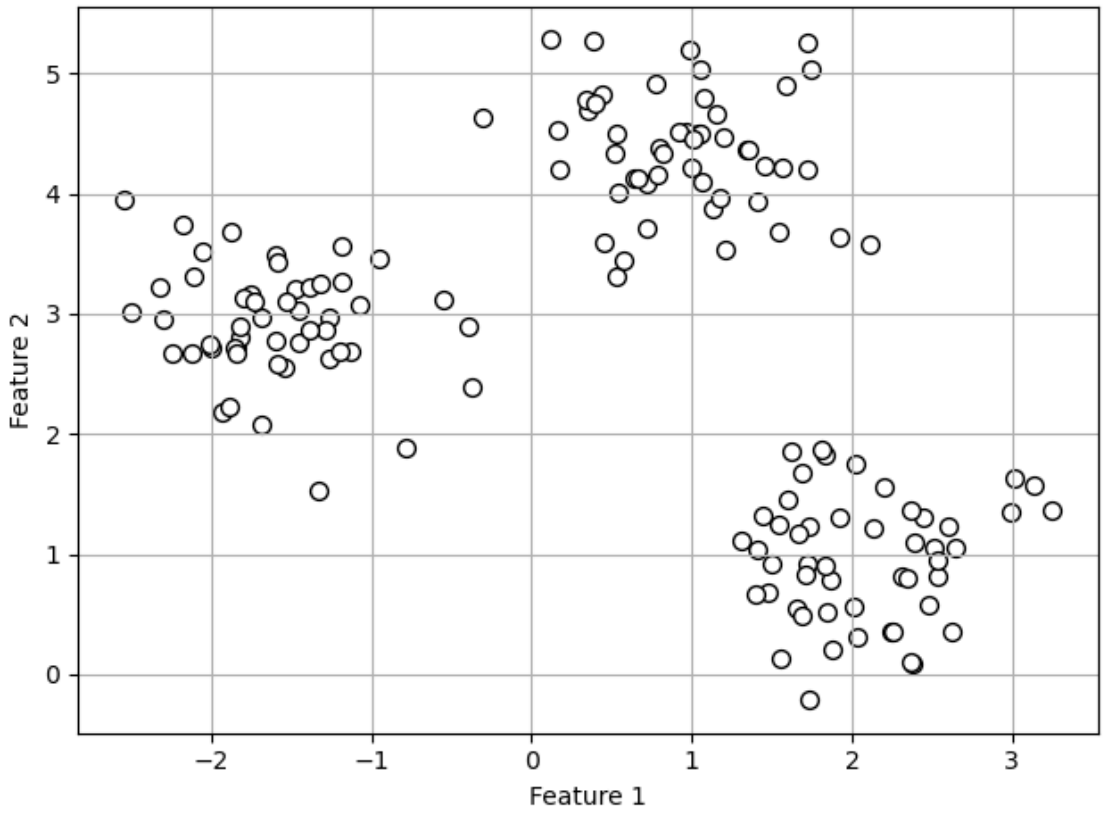
The 150 randomly generated points that are roughly grouped into three regions and two dimensions with higher density.

[1]:
```
from sklearn.datasets import make_blobs


X, y = make_blobs(n_samples=150,       # 150 randomly generated points
                  n_features=2,        # 2 inputs (dimension)
                  centers=3,           # 3 clusters (region)
```

```
                  cluster_std=0.5,     # density of samples
                  shuffle=True,
                  random_state=0)
```

Visualize the dataset in a scatter plot.

[2]:
```
import matplotlib.pyplot as plt


plt.scatter(X[:, 0], X[:, 1],
            c='white', marker='o', edgecolor='black', s=50)
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')

plt.grid()
plt.tight_layout()
#plt.savefig('figures/10_01.png', dpi=300)
plt.show()
```



Apply k-means algorithm to our example dataset using the `KMeans` class from scikit-learn's `cluster` module.

Problem: large `max_iter`

It is possible that k-means does not reach convergence for a particular run, which can be problematic (computationally expensive) if we choose relatively large values for `max_iter`.

Solution: large `tol`

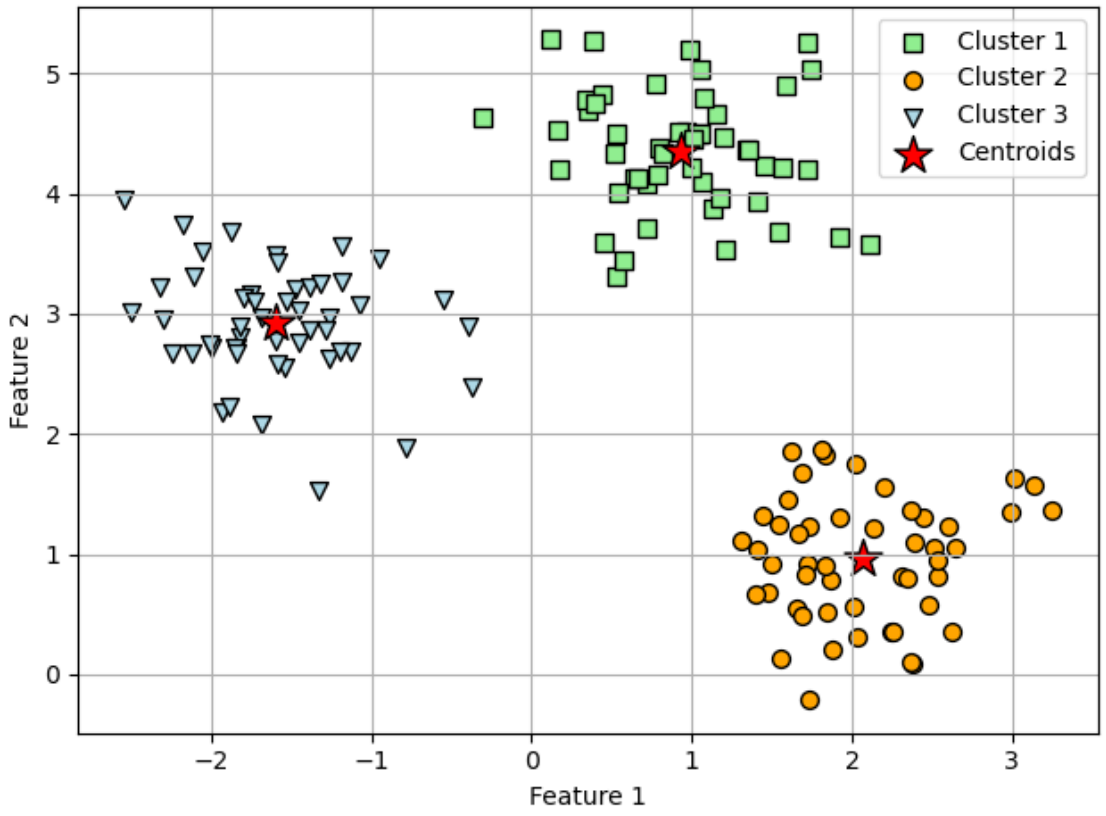
One way to deal with convergence problems is to choose larger values for `tol`, which is a parameter that controls the tolerance with regard to the changes in the within-cluster SSE to declare convergence. In the preceding code, we chose a tolerance of `1e-04` (=0.0001).

[3]:
```
from sklearn.cluster import KMeans


km = KMeans(n_clusters=3,        # specify the number of clusters a priori
            init='random',       # default is 'k-means++'
            n_init=10,           # run algorithms 10 times independently
            max_iter=300,
            tol=1e-04,
            random_state=0)

y_km = km.fit_predict(X)         # predicted cluster labels
```

Visualize the clusters that k-means identified in the dataset together with the cluster centroids. These are stored under the `cluster_centers_` attribute of the fitted `KMeans` object.

[4]:
```
plt.scatter(X[y_km == 0, 0],                # cluster 1
            X[y_km == 0, 1],
            s=50, c='lightgreen',
            marker='s', edgecolor='black',
            label='Cluster 1')
plt.scatter(X[y_km == 1, 0],                # cluster 2
            X[y_km == 1, 1],
            s=50, c='orange',
            marker='o', edgecolor='black',
            label='Cluster 2')
plt.scatter(X[y_km == 2, 0],                # cluster 3
            X[y_km == 2, 1],
            s=50, c='lightblue',
            marker='v', edgecolor='black',
            label='Cluster 3')
plt.scatter(km.cluster_centers_[:, 0],      # centroids
            km.cluster_centers_[:, 1],
            s=250, marker='*',
            c='red', edgecolor='black',
            label='Centroids')

plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
```

```
plt.legend(scatterpoints=1)
plt.grid()
plt.tight_layout()
#plt.savefig('figures/10_02.png', dpi=300)
plt.show()
```



1.2.2 10.1.2.A smarter way of placing the initial cluster centroids using k-means++

To use k-means++ with scikit-learn's `KMeans` object, we just need to set the `init` parameter to `'k-means++'`, which is strongly recommended in practice.

1.2.3 10.1.3.Hard versus soft clustering

Unfortunately, the FCM algorithm is not implemented in scikit-learn currently, but interested readers can try out the FCM implementation from the scikit-fuzzy package, which is available at https://github.com/scikit-fuzzy/scikit-fuzzy.

1.2.4 10.1.4.Using the elbow method to find the optimal number of clusters

Conveniently, we don't need to compute the within-cluster SSE explicitly when we are using scikit-learn, as it is already accessible via the `inertia_attribute` after fitting a KMeans model.

From previous k-means result, we have the following distortion:

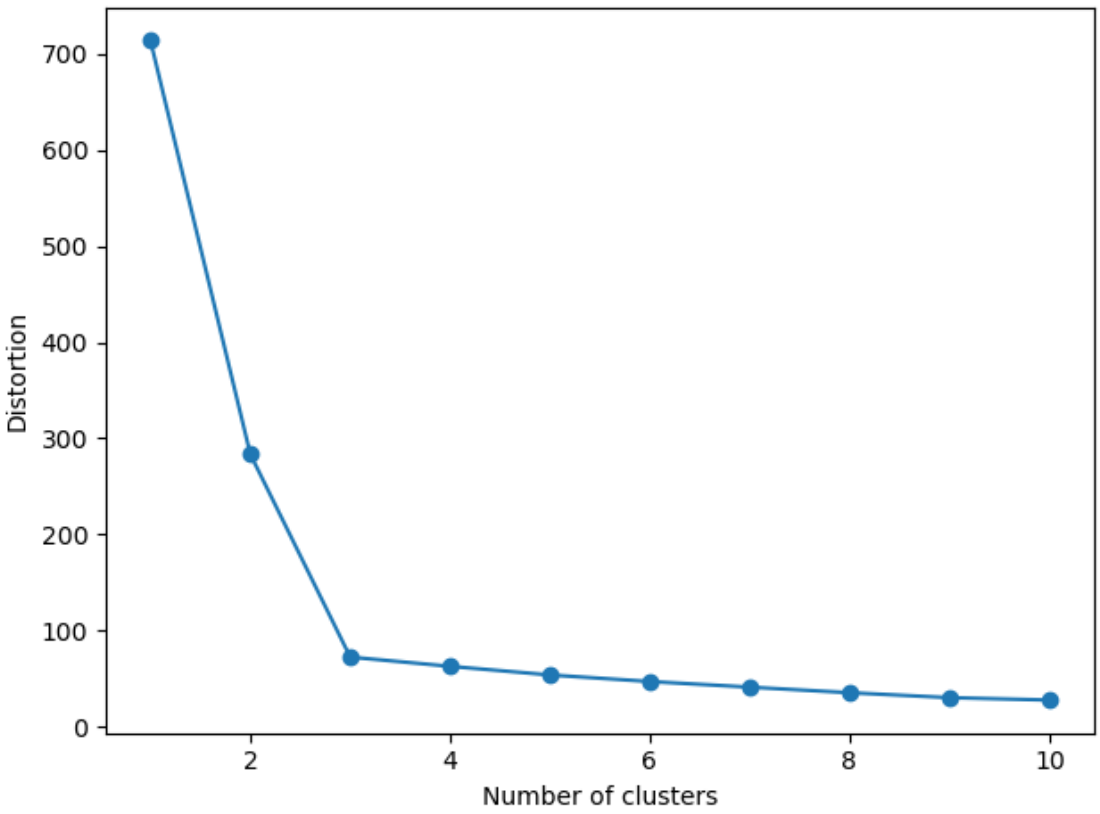
[5]:
```
print(f'Distortion: {km.inertia_:.2f}')
```

```
Distortion: 72.48
```

Based on the within-cluster SSE, we can use elbow method to estimate the optimal number of clusters k for a given task. And recall that the lower value of inertia (= lesser distortion) indicates better clustering.

In the following plot, the elbow is located at k = 3, so this is supporting evidence that k = 3 is a good choice for this dataset.

[6]:
```
# Calculate distortions with different number of clusters k
distortions = []
for i in range(1, 11):
    km = KMeans(n_clusters=i,
                init='k-means++',
                n_init=10,
                max_iter=300,
                random_state=0)
    km.fit(X)                                       # fit kmeans algorithm to data␣
 ↪for training
    distortions.append(km.inertia_)

# Visualize
plt.plot(range(1, 11), distortions, marker='o')
plt.xlabel('Number of clusters')
plt.ylabel('Distortion')
plt.tight_layout()
#plt.savefig('figures/10_03.png', dpi=300)
plt.show()
```



1.2.5 10.1.5.Quantifying the quality of clustering via silhouette plots

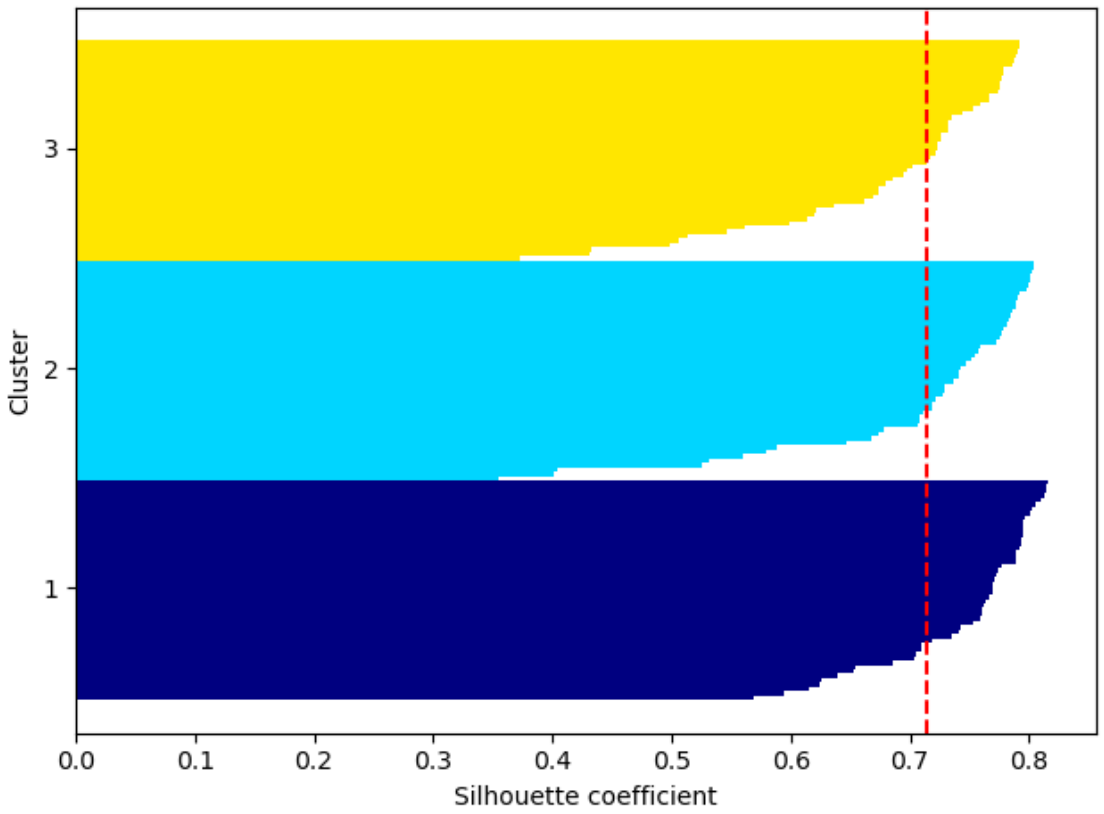
The silhouette coefficient is available as `silhouette_samples` from scikit-learn's metric module, and optionally, the `silhouette_scores` function can be imported for convenience. The `silhouette_scores` function calculates the average silhouette coefficient across all examples, which is equivalent to `numpy.mean(silhouette_samples(...))`.

There are two examples in the following sections: good and bad clustering.

1) Ex: good clustering (three centroids) - Silhouette coefficients are not close to 0 - Silhouette coefficients are approximately equally far away from the average silhouette coefficients (dotted line in the following figure)

[7]:
```
import numpy as np
from matplotlib import cm
from sklearn.metrics import silhouette_samples

# K-means clustering (3 clusters)
km = KMeans(n_clusters=3,
            init='k-means++',
            n_init=10,
            max_iter=300,
```

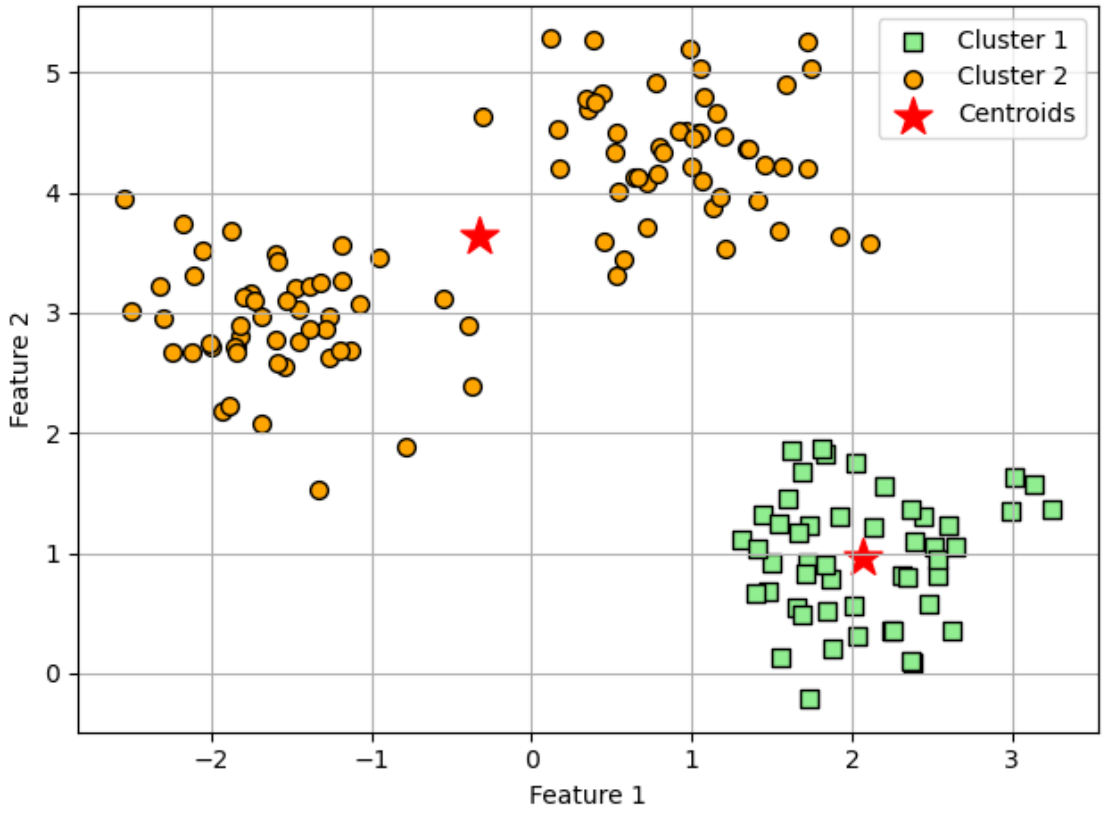
```
            tol=1e-04,
            random_state=0)
y_km = km.fit_predict(X)

# Calculate silhouette coefficient (values)
cluster_labels = np.unique(y_km)
n_clusters = cluster_labels.shape[0]
silhouette_vals = silhouette_samples(X, y_km, metric='euclidean')
y_ax_lower, y_ax_upper = 0, 0
yticks = []
for i, c in enumerate(cluster_labels):
    c_silhouette_vals = silhouette_vals[y_km == c]
    c_silhouette_vals.sort()
    y_ax_upper += len(c_silhouette_vals)
    color = cm.jet(float(i) / n_clusters)
    plt.barh(range(y_ax_lower, y_ax_upper), c_silhouette_vals, height=1.0,
             edgecolor='none', color=color)

    yticks.append((y_ax_lower + y_ax_upper) / 2.)
    y_ax_lower += len(c_silhouette_vals)

# Calculate average silhouette value (red dotted line in plot)
silhouette_avg = np.mean(silhouette_vals)
plt.axvline(silhouette_avg, color="red", linestyle="--")

# Silhouette plot
plt.yticks(yticks, cluster_labels + 1)
plt.ylabel('Cluster')
plt.xlabel('Silhouette coefficient')
plt.tight_layout()
#plt.savefig('figures/10_04.png', dpi=300)
plt.show()
```



2.1) Ex: bad clustering (two centroids) scatter plot - One of the centroids falls between two of the three spherical groupings of the input data.

[8]:
```
# K-means clustering (2 clusters)
km = KMeans(n_clusters=2,
            init='k-means++',
            n_init=10,
            max_iter=300,
            tol=1e-04,
            random_state=0)
y_km = km.fit_predict(X)

# Scatter plot
plt.scatter(X[y_km == 0, 0],
            X[y_km == 0, 1],
            s=50,
            c='lightgreen',
            edgecolor='black',
            marker='s',
            label='Cluster 1')
plt.scatter(X[y_km == 1, 0],
```

```
            X[y_km == 1, 1],
            s=50,
            c='orange',
            edgecolor='black',
            marker='o',
            label='Cluster 2')

plt.scatter(km.cluster_centers_[:, 0], km.cluster_centers_[:, 1],
            s=250, marker='*', c='red', label='Centroids')

plt.xlabel('Feature 1')
plt.ylabel('Feature 2')

plt.legend()
plt.grid()
plt.tight_layout()
#plt.savefig('figures/10_05.png', dpi=300)
plt.show()
```

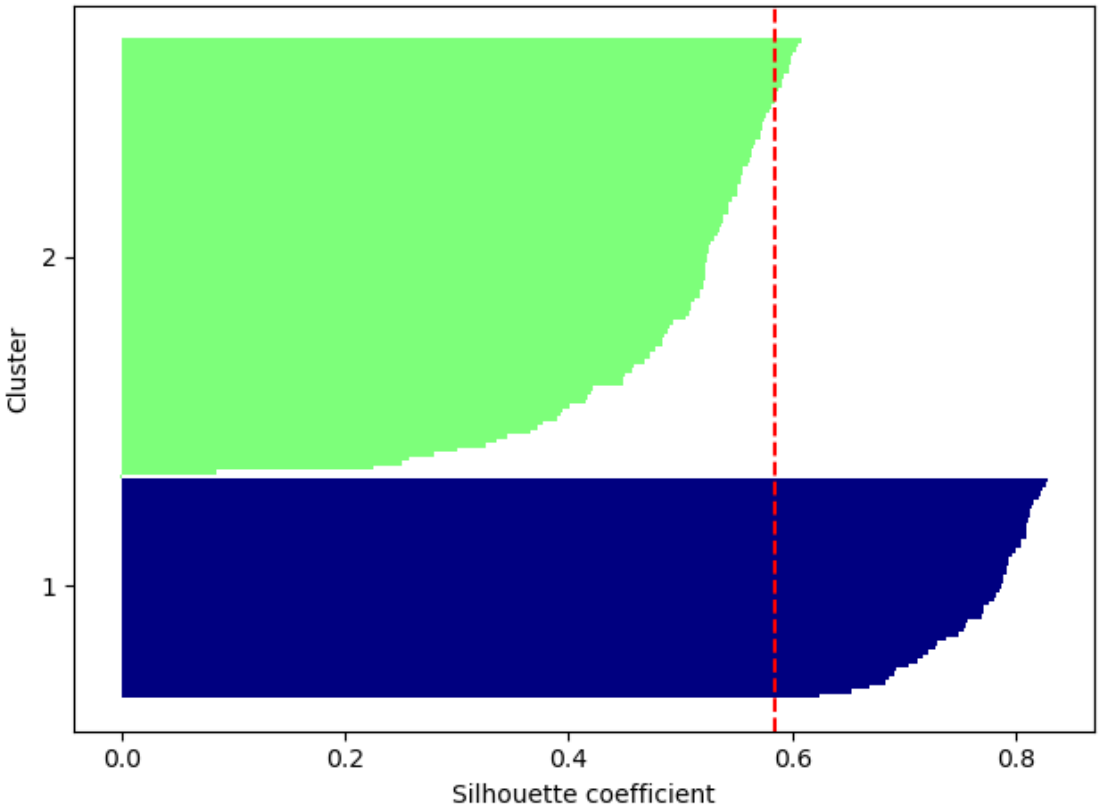


2.2) Ex: bad clustering (two centroids) silhouette plot

Typically we don't have the luxury of visualizing datasets in two-dimensional scatterplots in real-

world problems (data in higher dimensions). Thus, it's better for to use silhouette plot for evaluating the clustering results. - Silhouette coefficient of cluster 2 is closer to 0 - Silhouette coefficients are not equally far away from the average silhouette coefficients (dotted line in the following figure)

[9]:
```
# Calculate average silhouette value (red dotted line in plot)
cluster_labels = np.unique(y_km)
n_clusters = cluster_labels.shape[0]
silhouette_vals = silhouette_samples(X, y_km, metric='euclidean')
y_ax_lower, y_ax_upper = 0, 0
yticks = []
for i, c in enumerate(cluster_labels):
    c_silhouette_vals = silhouette_vals[y_km == c]
    c_silhouette_vals.sort()
    y_ax_upper += len(c_silhouette_vals)
    color = cm.jet(float(i) / n_clusters)
    plt.barh(range(y_ax_lower, y_ax_upper), c_silhouette_vals, height=1.0,
             edgecolor='none', color=color)

    yticks.append((y_ax_lower + y_ax_upper) / 2.)
    y_ax_lower += len(c_silhouette_vals)

# Calculate average silhouette value (red dotted line in plot)
silhouette_avg = np.mean(silhouette_vals)
plt.axvline(silhouette_avg, color="red", linestyle="--")

# Silhouette plot
plt.yticks(yticks, cluster_labels + 1)
plt.ylabel('Cluster')
plt.xlabel('Silhouette coefficient')
plt.tight_layout()
#plt.savefig('figures/10_06.png', dpi=300)
plt.show()
```



1.3 10.2.Organizing clusters as a hierarchical tree

1.3.1 10.2.1.Grouping clusters in bottom-up fashion

[10]:
```
Image(filename='./figures/10_07.png', width=400)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[10], line 1
----> 1 Image(filename='./figures/10_07.png', width=400)

NameError: name 'Image' is not defined
```

Generate a random data sample. - rows: different observations (IDs 0-4) - columns: different features (X, Y, Z) of those examples

[11]:
```
import pandas as pd
import numpy as np
```

```
np.random.seed(123)

variables = ['X', 'Y', 'Z']
labels = ['ID_0', 'ID_1', 'ID_2', 'ID_3', 'ID_4']

X = np.random.random_sample([5, 3])*10
df = pd.DataFrame(X, columns=variables, index=labels)
df
```

[11]:

	X	Y	Z
ID_0	6.964692	2.861393	2.268515
ID_1	5.513148	7.194690	4.231065
ID_2	9.807642	6.848297	4.809319
ID_3	3.921175	3.431780	7.290497
ID_4	4.385722	0.596779	3.980443

1.3.2 10.2.2.Performing hierarchical clustering on a distance matrix

Create a condensed distance matrix as input for the hierarchical clustering algorithm by `pdist` function from SciPy's `spatial.distance` submodule. And use `squareform` function to create a symmetrical pairwise distance matrix, shown in the following result.

[12]:
```
from scipy.spatial.distance import pdist, squareform


# Calculate Euclidean distance
row_dist = pd.DataFrame(squareform(pdist(df, metric='euclidean')),
                        columns=labels,
                        index=labels)
row_dist
```

[12]:

	ID_0	ID_1	ID_2	ID_3	ID_4
ID_0	0.000000	4.973534	5.516653	5.899885	3.835396
ID_1	4.973534	0.000000	4.347073	5.104311	6.698233
ID_2	5.516653	4.347073	0.000000	7.244262	8.316594
ID_3	5.899885	5.104311	7.244262	0.000000	4.382864
ID_4	3.835396	6.698233	8.316594	4.382864	0.000000

We can either pass a condensed distance matrix (upper triangular) from the `pdist` function, or we can pass the "original" data array and define the `metric='euclidean'` argument in `linkage` function in the following code. However, we should not pass the squareform distance matrix, which would yield different distance values although the overall clustering could be the same.

To sum it up, the three possible scenarios are listed here: 1. incorrect approach: Squareform distance matrix 2. correct approach: Condensed distance matrix 3. correct approach: Input matrix

[13]:
```
# 1. incorrect approach: Squareform distance matrix

from scipy.cluster.hierarchy import linkage


row_clusters = linkage(row_dist, method='complete', metric='euclidean')
pd.DataFrame(row_clusters,
             columns=['row label 1', 'row label 2',
                      'distance', 'no. of items in clust.'],
             index=[f'cluster {(i + 1)}'
                    for i in range(row_clusters.shape[0])])
```

```
/var/folders/vj/89w48r5119j30mvggnmk3czjf517hj/T/ipykernel_45578/1052818803.py:6
: ClusterWarning: scipy.cluster: The symmetric non-negative hollow observation
matrix looks suspiciously like an uncondensed distance matrix
  row_clusters = linkage(row_dist, method='complete', metric='euclidean')
```

[13]:

	row label 1	row label 2	distance	no. of items in clust.
cluster 1	0.0	4.0	6.521973	2.0
cluster 2	1.0	2.0	6.729603	2.0
cluster 3	3.0	5.0	8.539247	3.0
cluster 4	6.0	7.0	12.444824	5.0

[14]:
```
# 2. correct approach: Condensed distance matrix

row_clusters = linkage(pdist(df, metric='euclidean'), method='complete')
pd.DataFrame(row_clusters,
             columns=['row label 1', 'row label 2',
                      'distance', 'no. of items in clust.'],
             index=[f'cluster {(i + 1)}'
                    for i in range(row_clusters.shape[0])])
```

[14]:

	row label 1	row label 2	distance	no. of items in clust.
cluster 1	0.0	4.0	3.835396	2.0
cluster 2	1.0	2.0	4.347073	2.0
cluster 3	3.0	5.0	5.899885	3.0
cluster 4	6.0	7.0	8.316594	5.0

[15]:
```
# 3. correct approach: Input matrix

row_clusters = linkage(df.values, method='complete', metric='euclidean')
pd.DataFrame(row_clusters,
             columns=['row label 1', 'row label 2',
                      'distance', 'no. of items in clust.'],
             index=[f'cluster {(i + 1)}'
                    for i in range(row_clusters.shape[0])])
```

[15]:

	row label 1	row label 2	distance	no. of items in clust.
cluster 1	0.0	4.0	3.835396	2.0
cluster 2	1.0	2.0	4.347073	2.0
cluster 3	3.0	5.0	5.899885	3.0
cluster 4	6.0	7.0	8.316594	5.0

To take a closer look at the clustering results, we can turn those results into a pandas `DataFrame` as the following code.

As shown in the following result, the linkage matrix consists of several rows where each row represents one merge. - The first and second columns denote the most dissimilar members in each cluster - The third column reports the distance between those members - The last column returns the count of the members in each cluster

[16]:
```
>>> pd.DataFrame(row_clusters,
...              columns=['row label 1',
...                       'row label 2',
...                       'distance',
...                       'no. of items in clust.'],
...              index=[f'cluster {(i + 1)}' for i in
...                     range(row_clusters.shape[0])])
```

[16]:

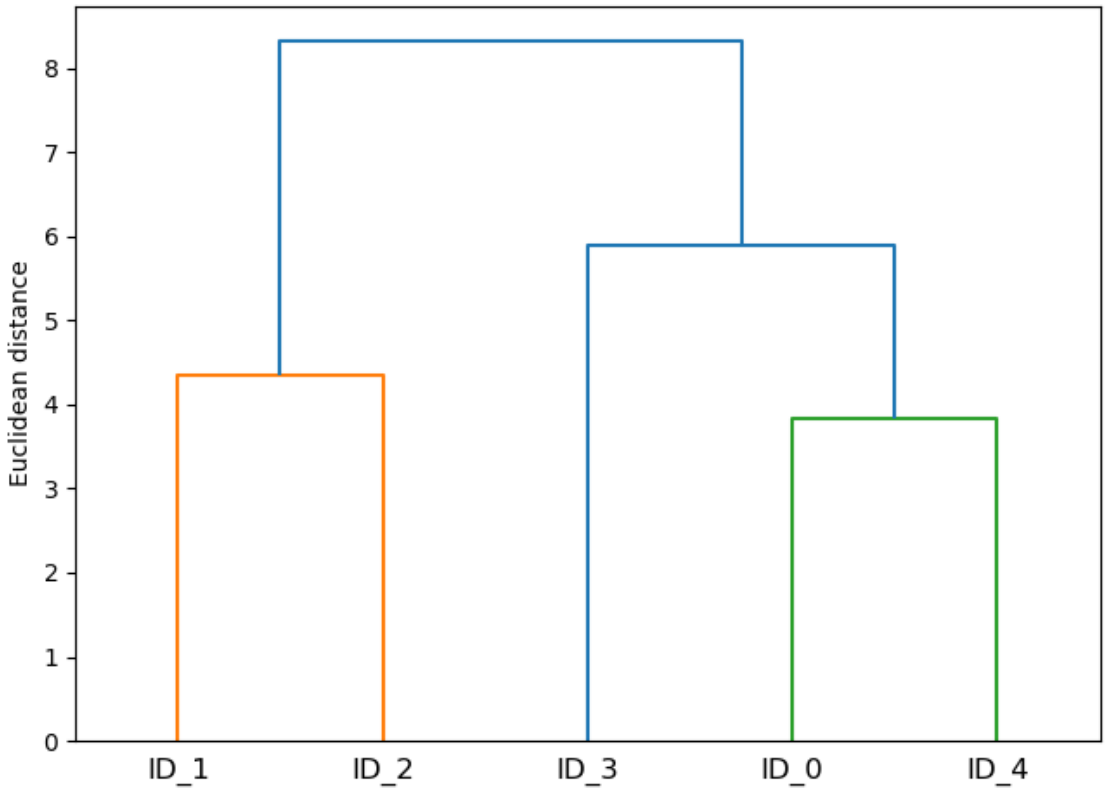
	row label 1	row label 2	distance	no. of items in clust.
cluster 1	0.0	4.0	3.835396	2.0
cluster 2	1.0	2.0	4.347073	2.0
cluster 3	3.0	5.0	5.899885	3.0
cluster 4	6.0	7.0	8.316594	5.0

Now that we have computed the linkage matrix, we can visualize the results in the form of a dendrogram.

Such a dendrogram summarizes the different clusters that were formed during the agglomerative hierarchical clustering; for example, you can see that the examples ID_0 and ID_4, followed by ID_1 and ID_2, are the most similar ones based on the Euclidean distance metric.

[17]:
```
from scipy.cluster.hierarchy import dendrogram


# make dendrogram black (part 1/2)
# from scipy.cluster.hierarchy import set_link_color_palette
# set_link_color_palette(['black'])

row_dendr = dendrogram(row_clusters,
                       labels=labels,
                       # make dendrogram black (part 2/2)
                       # color_threshold=np.inf
                       )
plt.tight_layout()
plt.ylabel('Euclidean distance')
```

```
#plt.savefig('figures/10_11.png', dpi=300,
#            bbox_inches='tight')
plt.show()
```



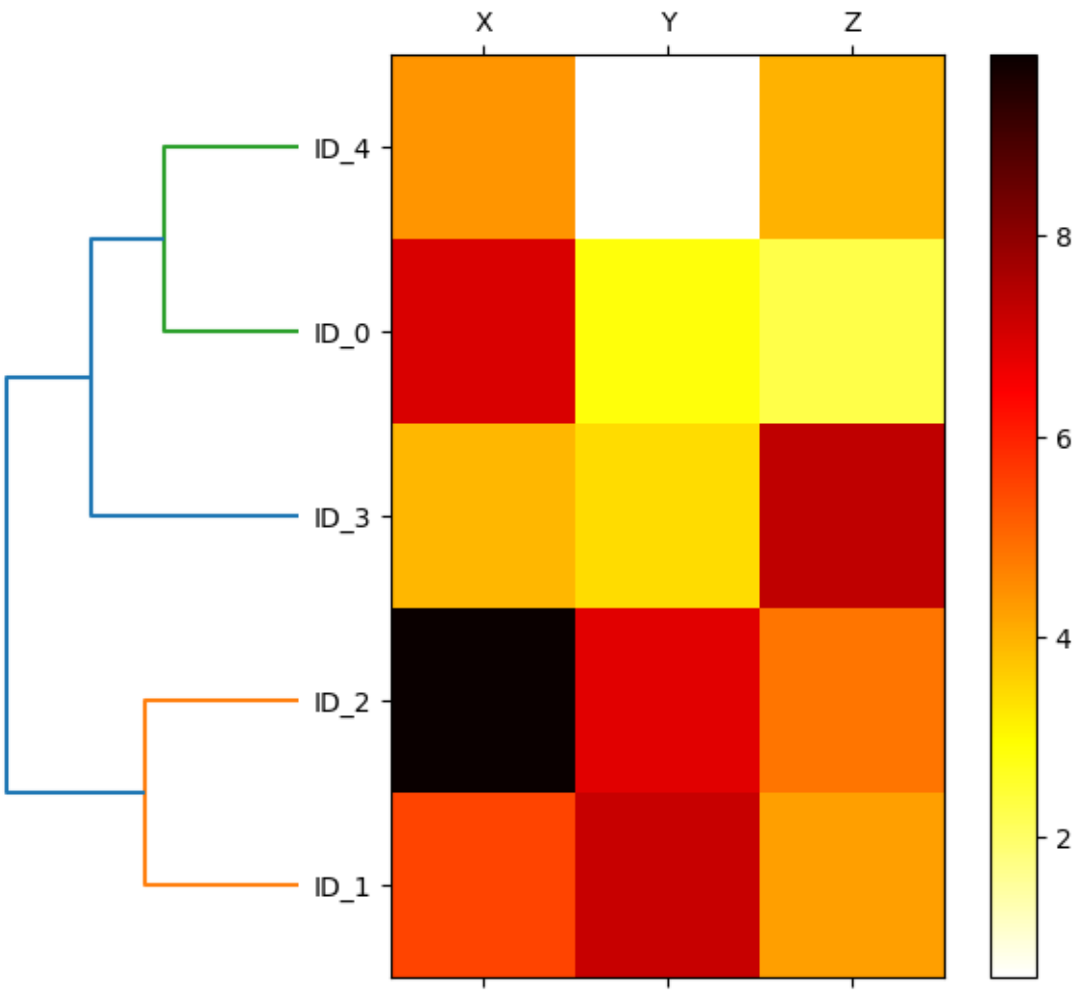
1.3.3 10.2.3.Attaching dendrograms to a heat map

Attach a dendrogram to a heat map with the following steps: 1. Create a new figure object and define the x-axis position, y-axis position, width, and height of the dendrogram via the `add_axes` attribute. Furthermore, we rotate the dendrogram 90 degrees counterclockwise. 2. Reorder the data in our initial `DataFrame` according to the clustering labels that can be accessed from the dendrogram object, which is essentially a Python dictionary, via the leaves key. 3. Modify the aesthetics of the dendrogram by removing the axis ticks and hiding the axis spines. 4. Construct the heat map from the reordered `DataFrame` and position it next to the dendrogram. 5. Add a color bar and assign the feature and data record names to the x and y axis tick labels.

[18]:
```
# 1. Plot row dendrogram
fig = plt.figure(figsize=(8, 8), facecolor='white')
axd = fig.add_axes([0.09, 0.1, 0.2, 0.6])                    # define the␣
 ↪attributes of the dendrogram
row_dendr = dendrogram(row_clusters, orientation='left')  # note: for␣
 ↪matplotlib < v1.5.1, use orientation='right'
```

```
# 2. Reorder data with respect to clustering
df_rowclust = df.iloc[row_dendr['leaves'][::-1]]
axd.set_xticks([])
axd.set_yticks([])

# 3. Remove axes spines from dendrogram
for i in axd.spines.values():
    i.set_visible(False)

# 4. Construct and place heatmap
axm = fig.add_axes([0.23, 0.1, 0.6, 0.6])  # x-pos, y-pos, width, height
cax = axm.matshow(df_rowclust, interpolation='nearest', cmap='hot_r')

# 5. Plot heatmap
fig.colorbar(cax)
axm.set_xticklabels([''] + list(df_rowclust.columns))
axm.set_yticklabels([''] + list(df_rowclust.index))
#plt.savefig('figures/10_12.png', dpi=300)
plt.show()
```

```
/var/folders/vj/89w48r5119j30mvggnmk3czjf517hj/T/ipykernel_45578/73672617.py:21:
UserWarning: FixedFormatter should only be used together with FixedLocator
  axm.set_xticklabels([''] + list(df_rowclust.columns))
/var/folders/vj/89w48r5119j30mvggnmk3czjf517hj/T/ipykernel_45578/73672617.py:22:
UserWarning: FixedFormatter should only be used together with FixedLocator
  axm.set_yticklabels([''] + list(df_rowclust.index))
```



1.3.4 10.2.4.Applying agglomerative clustering via scikit-learn

This `AgglomerativeClustering` function in scikit-learn allows us to choose the number of clusters that we want to return, which is useful if we want to prune the hierarchical cluster tree.

In this case, we set number of clusters as `n_clusters=3`

From the result `Cluster labels: [1 0 0 2 1]`, first entity in the vector represents ID_0 belongs to label 1, vice versa. With such, we can read the result `Cluster labels: [1 0 0 2 1]` as follows: - ID_0 and ID_4 belongs to label 1 - ID_1 and ID_2 belongs to label 0 - ID_3 belongs to label 2

[24]:
```
from sklearn.cluster import AgglomerativeClustering

# Initialize the AgglomerativeClustering model
ac = AgglomerativeClustering(n_clusters=3,
                             affinity='euclidean',
                             linkage='complete')
```

```
# Fit the model and predict labels
labels = ac.fit_predict(X)  # Make sure X is your data matrix

# Print the cluster labels
print(f'Cluster labels: {labels}')
```

```
Cluster labels: [1 0 0 2 1]
```

Note, the original code from Sebastian Raschka below might not work depending on the versions of scikit-learn you are running. I have upfated my scikit-learn and the new version uses 'affinity' for AggromerativeClustering

```
%This is the original code in the repository

 from sklearn.cluster import AgglomerativeClustering


 ac = AgglomerativeClustering(n_clusters=3,
                              metric='euclidean',
                              linkage='complete')
 labels = ac.fit_predict(X)
 print(f'Cluster labels: {labels}')
```

Now, change number of clusters to `n_clusters=2`, we can see that - ID_0, ID_3 and ID_4 belongs to label 0 - ID_1 and ID_2 belongs to label 1

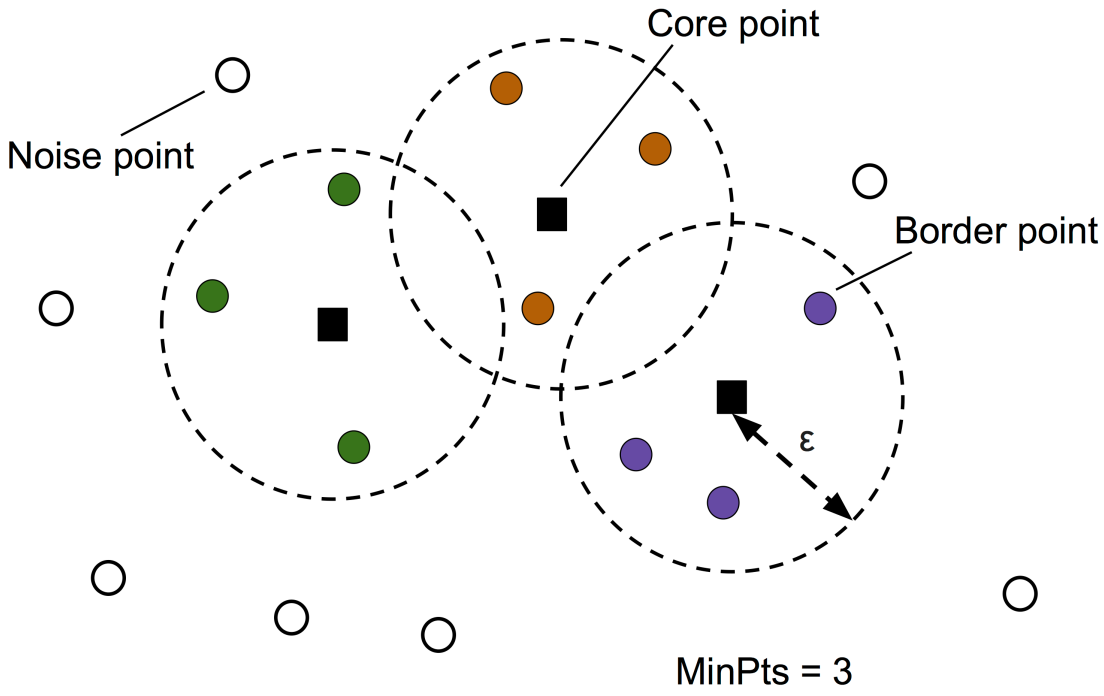
[25]:
```
ac = AgglomerativeClustering(n_clusters=2,
                             affinity='euclidean',
                             linkage='complete')
labels = ac.fit_predict(X)
print(f'Cluster labels: {labels}')
```

```
Cluster labels: [0 1 1 0 0]
```

1.4 10.3.Locating regions of high density via DBSCAN

[28]:
```
Image(filename='figures/10_13.png', width=500)
```

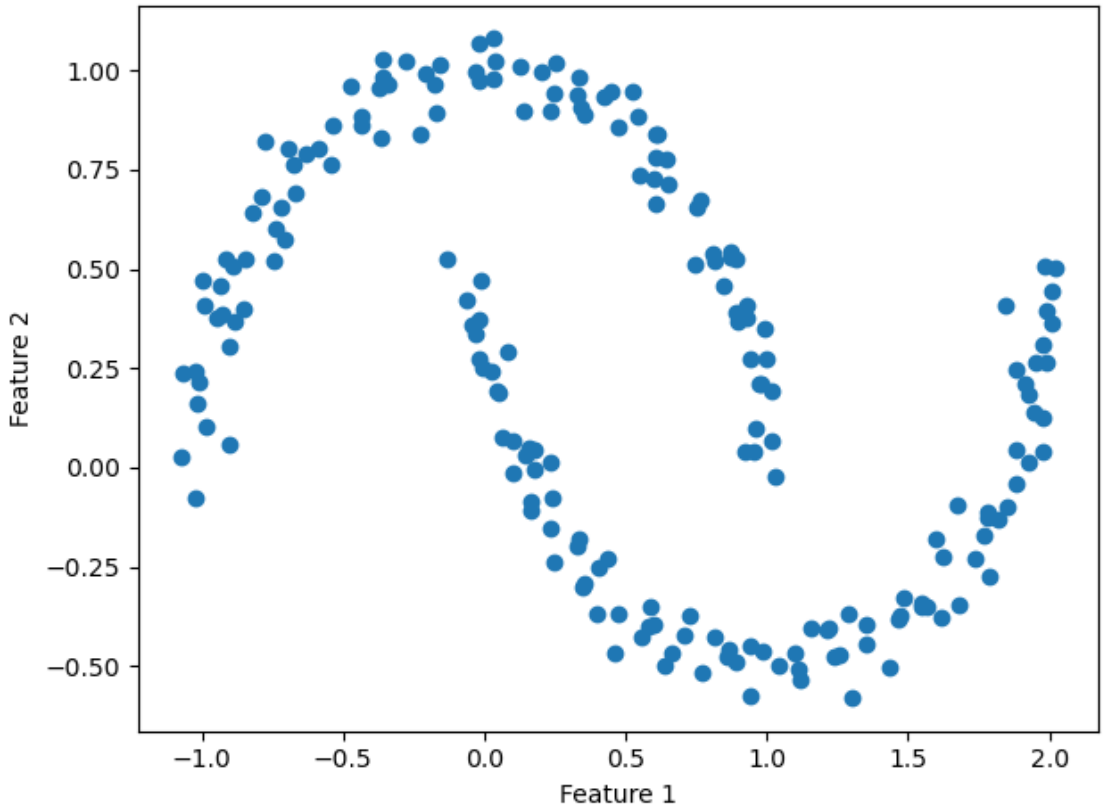
[28]:



In this section, we will compare three different clustering approaches: k-means, ACL, DBSCAN, with a dataset of half-moon-shaped structures. *(recall that ACL = Agglomerative Complete Linkage)*

First, let's create the dataset with the build-in dataset in scikit-learn.

[30]:
```
from sklearn.datasets import make_moons


X, y = make_moons(n_samples=200, noise=0.05, random_state=0)
plt.scatter(X[:, 0], X[:, 1])

plt.xlabel('Feature 1')
plt.ylabel('Feature 2')

plt.tight_layout()
#plt.savefig('figures/10_14.png', dpi=300)
plt.show()
```

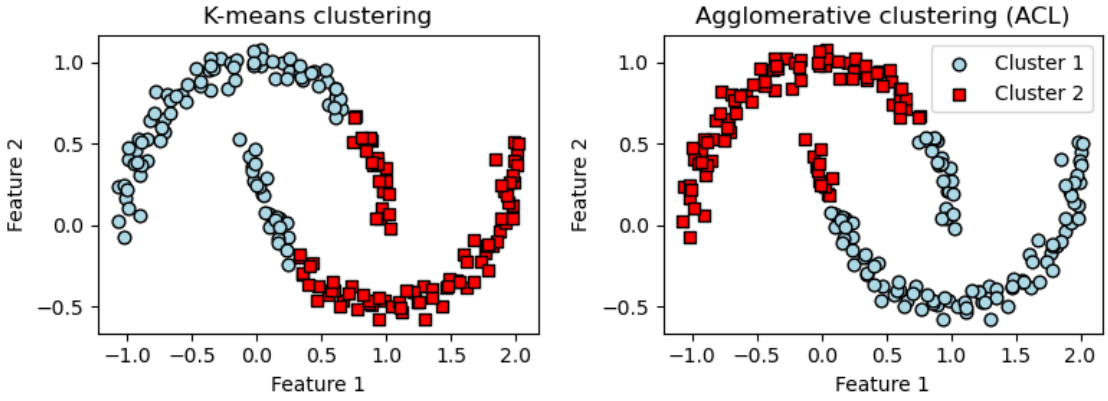


1) K-means and ACL algorithm

Based on the visualized clustering results, we can see that the k-means algorithm was unable to separate the two clusters, and also, the hierarchical clustering algorithm (ACL) was challenged by those complex shapes.

[32]:
```
f, (ax1, ax2) = plt.subplots(1, 2, figsize=(8, 3))

# 1. K-means clustering
km = KMeans(n_clusters=2, random_state=0)
y_km = km.fit_predict(X)
ax1.scatter(X[y_km == 0, 0], X[y_km == 0, 1],
            edgecolor='black',
            c='lightblue', marker='o', s=40, label='cluster 1')
ax1.scatter(X[y_km == 1, 0], X[y_km == 1, 1],
            edgecolor='black',
            c='red', marker='s', s=40, label='cluster 2')
ax1.set_title('K-means clustering')
ax1.set_xlabel('Feature 1')
ax1.set_ylabel('Feature 2')

# 2. ACL clustering
```

```
ac = AgglomerativeClustering(n_clusters=2,
                             affinity='euclidean',
                             linkage='complete')
y_ac = ac.fit_predict(X)
ax2.scatter(X[y_ac == 0, 0], X[y_ac == 0, 1], c='lightblue',
            edgecolor='black',
            marker='o', s=40, label='Cluster 1')
ax2.scatter(X[y_ac == 1, 0], X[y_ac == 1, 1], c='red',
            edgecolor='black',
            marker='s', s=40, label='Cluster 2')
ax2.set_title('Agglomerative clustering (ACL)')
ax2.set_xlabel('Feature 1')
ax2.set_ylabel('Feature 2')

# Plot attributes
plt.legend()
plt.tight_layout()
#plt.savefig('figures/10_15.png', dpi=300)
plt.show()
```



2) DBSCAN

The DBSCAN algorithm can successfully detect the half-moon shapes, which highlights one of the strengths of DBSCAN—clustering data of arbitrary shapes.

[33]:
```
from sklearn.cluster import DBSCAN

# 3. DBSCAN clustering
db = DBSCAN(eps=0.2, min_samples=5, metric='euclidean')
y_db = db.fit_predict(X)
plt.scatter(X[y_db == 0, 0], X[y_db == 0, 1],
            c='lightblue', marker='o', s=40,
            edgecolor='black',
```

```
            label='Cluster 1')
plt.scatter(X[y_db == 1, 0], X[y_db == 1, 1],
            c='red', marker='s', s=40,
            edgecolor='black',
            label='Cluster 2')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')

# Plot attributes
plt.legend()
plt.tight_layout()
#plt.savefig('figures/10_16.png', dpi=300)
plt.show()
```